Deterministic simulations of nondeterministic and randomized time

The ETH counting algorithm and HPV blocking

30 October 2026

1 The SAT and counting algorithms

We begin with the algorithmic result from the ETH refutation. All logarithms in this note are base two.

Theorem 1.1 (OpenAI [1, Theorem 1.1]). *For each fixed $k \geq 2$, there are deterministic algorithms deciding k-SAT and exactly counting the satisfying assignments of a k-CNF with $N \geq 20$ variables and m clauses in time*

$$(N+m)^{O_k(1)} 2^{O_k\left(N \log\log N/\sqrt{\log N}\right)}$$

and space $(N+m)^{O_k(1)}$. For each fixed $D > 0$, on formulas with $m \leq DN$, the running time is

$$(N+m)^{O_{k,D}(1)} 2^{O_{k,D}\left(N/\sqrt{\log N}\right)}.$$

The counting algorithm gives the decision algorithm by testing whether the count is positive. We will use the tensor-contraction form of the counting algorithm, which also appears in the manuscript.

A *Boolean tensor network* consists of a graph G with one bit on each edge and a function F_v at each vertex, whose arguments are the bits on its incident edges. Parallel edges have distinct ports. Its contraction over a ring R is

$$Z(G) = \sum_{z \in \{0,1\}^{E(G)}} \prod_{v \in V(G)} F_v(z|_{\delta(v)}), \tag{1}$$

where $\delta(v)$ is the ordered list of edges incident to v.

Lemma 1.2 (Boolean tensor contraction [1, Theorem D.1]). *Let $s \geq 2$. A bipartite Boolean tensor network with $O(s)$ edges, polynomially many vertices, maximum degree $O(\log s)$, and explicit tensor tables of total length $\mathrm{poly}(s)$ can be contracted over $\mathbb{Z}/2^P\mathbb{Z}$, for $1 \leq P = O(s)$, in deterministic time*

$$\mathrm{poly}(s)\, 2^{O(s/\sqrt{\log s})}$$

and polynomial space. The constants depend only on the fixed constants in the size, degree, description and precision bounds.

Proof. Take $\ell = 1$ in Theorem D.1. For sufficiently large s, the degree bound $O(\log s)$ is at most $\lceil s^{9/10} \rceil$, and reading an entry from an explicit polynomial-size table meets the theorem's entry-computation bound. The finitely many smaller sizes are handled directly. □

2 HPV blocks and their counting extension

We work with multitape Turing machines having a fixed finite alphabet and a fixed number of one-dimensional tapes. It is convenient to use two-way infinite tapes, with all heads initially at position zero. The input tape is initialized to x and blanks; a read-only tape can be included by leaving its symbols unchanged.

We represent a computation by binary choices. At a deterministic step the choice bit is ignored. After halting, the machine stays in an absorbing accepting or rejecting state and ignores subsequent choices. For a probabilistic machine, this preserves its acceptance probability. A nondeterministic machine with constant branching can be put in this form with constant overhead by encoding each transition choice in binary and rejecting invalid encodings.

The following is a convenient form of the blocking idea of Hopcroft, Paul and Valiant [2]; see also [3, Lemma 2.1 and Section 3.2]. Using three neighboring tape blocks lets us apply the construction directly to the original machine.

Lemma 2.1 (HPV block decomposition)**.** *Fix a k-tape machine M, an input x, and an integer time bound $t \geq \max\{|x|, 2\}$. For $2 \leq b \leq t$, put*

$$q = \lceil t/b \rceil, \qquad \tau = qb,$$

and pad every run to τ steps. The runs can be partitioned by at most 3^{kq} movement schedules. For any fixed schedule, the run is described by q local transitions, each on $O_M(b)$ bits, together with $O_M(q)$ word connections of length $O_M(b)$ between transitions and their initial or final conditions. The connections are determined by the schedule and can be constructed in polynomial time.

Proof. Divide time into q intervals of b steps and each tape into blocks

$$B_j = \{jb, jb+1, \ldots, (j+1)b-1\}, \qquad j \in \mathbb{Z}.$$

At the start of interval i, write the position of head h as $p_{h,i} = bj_{h,i} + r_{h,i}$, where $0 \leq r_{h,i} < b$. A head makes at most b moves during the interval, so

$$j_{h,i+1} - j_{h,i} \in \{-1, 0, 1\}.$$

A schedule specifies these displacements for every tape and interval, including the last interval. Since $j_{h,0} = 0$, it determines all the block indices $j_{h,i}$. There are at most 3^{kq} schedules, and every run has exactly one.

During interval i, head h stays in the three blocks

$$B_{j_{h,i}-1}, \quad B_{j_{h,i}}, \quad B_{j_{h,i}+1}.$$

Indeed, starting in $[bj, bj+b-1]$, it remains in $[bj-b, bj+2b-1]$ after at most b moves. Thus the interval is determined by its b choice bits, its entering state and head offsets, and the contents of these three blocks on each tape. Its leaving data consist of the updated contents, the state and the head offsets; the leaving block indices must agree with the schedule. Each block has $O_M(b)$ bits, while the state and offsets use $O_M(\log b)$ bits.

For each physical tape block, order the intervals whose three-block neighborhoods include it. Connect its contents after one appearance to its contents before the next. Its first contents are fixed by x and the initial blanks, and its final contents are unrestricted. Between appearances the block is never touched. Within an interval, all cells that are not touched must be copied unchanged.

Connect the state and head offsets along the sequence of time intervals. Fix the initial state and offsets and impose the desired condition on the final state. There are $3kq$ tape-block appearances and q control records, so this uses $O_M(q)$ word connections, each of length $O_M(b)$. The schedule supplies the absolute block indices; only the within-block offsets are carried in the control records. Constructing the ordered lists and their connections takes polynomial time. □

Lemma 2.2 (Block counting)**.** *In the setting of Lemma 2.1, suppose every run of $M(x)$ halts within t steps. There is a family of at most 3^{kq} bipartite Boolean tensor networks G_σ such that*

$$\sum_\sigma Z(G_\sigma) = A_M(x) := \big|\{y \in \{0,1\}^\tau : M(x;y) \text{ accepts}\}\big|, \tag{2}$$

where the contractions in this identity are over the integers.

Each network has $O_M(t)$ Boolean edges and vertices, maximum degree $O_M(b)$, and entries in $\{0,1,\dots,2^b\}$. Its graph and uniform entry procedures can be constructed in polynomial time. An entry is computable in $2^b \operatorname{poly}_M(t)$ time and polynomial space; all its tensor tables can be written explicitly in $\operatorname{poly}_M(t)2^{O_M(b)}$ time and space.

Proof. Fix a schedule σ. For each interval i, make a tensor whose ports are its entering and leaving data from Lemma 2.1. Given values for all its ports, let its entry be

$$F_i(\text{ports}) = \big|\{u \in \{0,1\}^b : \text{the run with choices } u \text{ matches the ports and } \sigma\}\big|. \tag{3}$$

Here the local simulation checks the outgoing block contents, the state and offsets, and the prescribed head displacements. In particular, untouched cells must retain their incoming values. Use fixed injective encodings of symbols, states and offsets, and give entry zero to any invalid encoding.

Join these tensors using the word connections in Lemma 2.1, with one Boolean edge per bit. Pin the initial words to their known values. At the end, sum over the final tape contents and head offsets, and require the state to be accepting. These conditions are implemented by 0–1 tensors. Subdivide every edge by an equality tensor $E(a,c) = \mathbf{1}[a = c]$. This makes the graph bipartite and preserves the contraction, since each edge assignment has exactly one consistent extension.

To check the counting identity, expand each entry in (3) as a sum of indicators indexed by $u_i \in \{0,1\}^b$. A nonzero term in the contraction then specifies consistent boundary data and a tuple $(u_0,\dots,u_{q-1})$. Starting with the initial configuration, induction over the intervals shows that these data describe the actual run with choices $y = u_0u_1\cdots u_{q-1}$. The memory connections give each interval the correct contents from a block's previous appearance, or from initialization if it has not appeared before. The local checks force its subsequent contents and its head displacements.

Conversely, an accepting run with schedule σ fixes every boundary word and every subdivision bit uniquely. This includes unused side blocks, whose initial contents are fixed and whose untouched cells are copied. It therefore contributes exactly one expanded term. Several choice strings may have the same boundary data; their multiplicity is retained by the integer entry (3). Thus $Z(G_\sigma)$ counts precisely the accepting strings with schedule σ. Summing over schedules gives (2).

There are q interval tensors, each of degree $O_M(b)$. The initial and final conditions and the subdivisions change the size by at most a constant factor. Hence there are $O_M(qb) = O_M(t)$ edges and vertices, since $\tau = qb \le t + b \le 2t$, and the maximum degree is $O_M(b)$. Each entry is at most 2^b. Enumerating the b choice bits and simulating the interval computes it in $2^b \operatorname{poly}_M(t)$ time and polynomial space. A tensor of degree $O_M(b)$ has $2^{O_M(b)}$ entries, which gives the stated bound for constructing all tables. □

3 Deterministic simulations

Theorem 3.1. *Assume the counting algorithm in Lemma 1.2. For every time-constructible function $T(n) \geq \max\{n, 2\}$,*

$$\mathsf{NTIME}[T] \subseteq \mathsf{DTIME}\left[2^{O(T/\sqrt{\log T})}\right],$$
$$\mathsf{BPTIME}[T] \subseteq \mathsf{DTIME}\left[2^{O(T/\sqrt{\log T})}\right].$$

Both simulations use polynomial space in T. In fact, the algorithm computes the exact number of accepting padded binary choice strings.

Proof. Fix a machine M witnessing either time bound and an input x of length n. Normalize M as above and choose a fixed integer c_M such that $t = c_M T(n)$ bounds every run. Time constructibility allows us to compute this bound. Set

$$b = \lceil \log(t+2) \rceil, \qquad q = \lceil t/b \rceil, \qquad \tau = qb.$$

Then $b = \Theta(\log t)$, $q = O(t/\log t)$ and $\tau = \Theta(t)$.

Apply Lemma 2.2. For each schedule, its network has $O_M(t)$ Boolean edges, maximum degree $O_M(\log t)$, and explicit tables of total length $\mathrm{poly}_M(t)$, constructible in polynomial time. Reduce the entries modulo $2^{\tau+1}$ and apply Lemma 1.2 with $s = t$ and $P = \tau + 1 = O(t)$. Each contraction takes

$$\mathrm{poly}_M(t)\, 2^{O_M(t/\sqrt{\log t})}$$

time and polynomial space.

Add all the contractions modulo $2^{\tau+1}$. By (2), the result is $A_M(x)$ modulo $2^{\tau+1}$. Since

$$0 \leq A_M(x) \leq 2^{\tau} < 2^{\tau+1},$$

its least nonnegative residue is the exact integer count. There are $3^{kq} = 2^{O_M(t/\log t)}$ schedules, so the total time is

$$\begin{aligned} 3^{kq}\, \mathrm{poly}_M(t)\, 2^{O_M(t/\sqrt{\log t})} &= 2^{O_M(t/\log t) + O_M(\log t) + O_M(t/\sqrt{\log t})} \\ &= 2^{O_M(t/\sqrt{\log t})} \\ &= 2^{O_M(T(n)/\sqrt{\log T(n)})}. \end{aligned}$$

Process the schedules one at a time, keeping one network, its contraction workspace and an accumulator of $\tau + 1$ bits. The resulting space bound is $\mathrm{poly}_M(T(n))$. The bit-operation bound gives the same exponential bound on a deterministic multitape Turing machine: polynomial simulation overhead is absorbed into the constant in the exponent.

For the nondeterministic case,

$$x \in L \quad \Longleftrightarrow \quad A_M(x) > 0.$$

Testing the computed count for positivity proves the first containment.

For the probabilistic case, the normalized machine uses a uniform string $y \in \{0,1\}^{\tau}$, including the ignored bits at deterministic and padded steps. Hence its acceptance probability is $A_M(x)/2^{\tau}$. Bounded error gives

$$x \in L \implies A_M(x) \geq \tfrac{2}{3}\, 2^{\tau}, \qquad x \notin L \implies A_M(x) \leq \tfrac{1}{3}\, 2^{\tau}.$$

Accepting exactly when $2A_M(x) > 2^{\tau}$ therefore decides L deterministically and proves the second containment. □

The same exact-counting argument applies to strict-majority probabilistic computation: comparing the count with $2^{\tau-1}$ also gives the corresponding simulation of $\mathsf{PPTIME}[T]$.

References

[1] OpenAI. *The Exponential Time Hypothesis is False*. 17 October 2026, revised 25 October 2026. Theorem 1.1 and Appendix D, Theorem D.1. Manuscript.

[2] John Hopcroft, Wolfgang Paul and Leslie Valiant. *On Time Versus Space*. Journal of the ACM **24**(2), 332–337, 1977. Article.

[3] Ryan Williams. *Simulating Time With Square-Root Space*. STOC 2025; full version dated 25 February 2025. Lemma 2.1 and Section 3.2. Full version.